



Universidad Politécnica de Querétaro

Ingeniería en Sistemas computacionales

Materia: Programación web

“Documento de planificación de Sprints - Mercy Food”

Docente: José Javier Moya Moya

Por: Andrés Joshua León Barranco, Alberto Luna Rufino, Eduardo Loredo García, Jorge Armando López Morales, Diego Jiménez Vargas y Saúl Alejandro Maldonado Ladrillero

Grupo: S-204

10/nov/2025

1. Estructura de Git/GitHub (Estrategia y roles)

- Estrategia de Branching: Se utilizará Gitflow.
 1. main: Rama principal (estable). Se prohíben *commits* directos.
 2. develop: Rama de integración. Todas las tareas se fusionan aquí mediante Pull Requests (PRs).
 3. feature/HU_X.X-descripcion: Ramas de tareas. Se crean desde develop y se fusionan de nuevo a develop.
- Horarios y proceso de Pull Requests:
 1. Los PRs de feature a develop deben crearse al finalizar cada historia de usuario.
 2. Proceso de code review: El PR debe ser revisado y aprobado por el "Responsable de Merge" del sprint actual antes de fusionarse.
 3. Horario límite para PRs: Todos los PRs del sprint deben estar en develop antes de las 18:00h del día anterior a la presentación del sprint.
- Responsables de Merge (Revisión y aprobación de PRs):
 - Sprint 1: Andrés Joshua León Barranco
 - Sprint 2: Alberto Luna Rufino
 - Sprint 3: Eduardo Loredó García
 - Sprint 4: Jorge Armando López Morales

2. Planificación de Sprints

Sprint 1: Planificación v2.0 (10–12 Nov)

Objetivo: Documentar el plan, configurar el entorno y realizar la investigación de las nuevas APIs y funcionalidades.

Historia de usuario (Tarea)	Asignación	Criterios de aprobación
HU_1.1: (Doc) Finalizar, imprimir y subir el "Documento de Planificación de Sprints".	Andrés J.	El documento está entregado en Classroom y en físico el 12 de Nov.
HU_1.2: (Git) Configurar el repositorio en GitHub (crear rama develop, proteger main).	Andrés J.	La rama main no permite pushes directos. La rama develop existe.
HU_1.3: (Investigación) Investigar y documentar la implementación de Laravel Socialite (Google).	Diego J.	Entregar un documento con los pasos, paquetes y configuraciones necesarias.
HU_1.4: (Investigación) Investigar y documentar la implementación de la API de PayPal Sandbox.	Saúl M.	Entregar un documento con los pasos, paquetes y lógica (crear/capturar orden) necesarios.
HU_1.5: (Investigación) Modelado de BD para un sistema de Reseñas (Restaurante y Repartidor).	Diego J.	Entregar un diagrama de BD de la nueva tabla reviews y sus relaciones.
HU_1.6: (Investigación) Auditar los Seeders actuales (UserSeeder, PlatilloSeeder) e identificar errores.	Saúl M.	Entregar un documento listando los errores y el plan para corregirlos.
HU_1.7: (Doc) Auditar el código existente y documentar la "deuda técnica".	Alberto, Eduardo, Jorge	Un documento que lista puntos de mejora en el código actual.

- Gestión de PRs (Sprint 1):
 - Fecha y hora del PR: miércoles, 12 de Nov, 12:00h.
 - Responsable de hacer el PR: Cada integrante (a develop).
 - Responsable de aprobar el PR: Andrés J.
 - Rama destino: develop

Sprint 2: Autenticación social y corrección de Seeders (13–19 Nov)

Objetivo: Implementar el login con Google y corregir los seeders para tener una base de datos de prueba funcional.

Historia de usuario (Tarea)	Asignación	Criterios de aprobación
HU_2.1: (Code) Implementar Login con Google (Socialite), basado en la doc del S1.	Diego J.	El usuario puede registrarse/loguearse con su cuenta de Google.
HU_2.2: (Code) Corregir y actualizar el UserSeeder (crear los 3 roles).	Saúl M.	El seeder crea Clientes, Restaurantes y Repartidores de prueba funcionales.
HU_2.3: (Code) Corregir y actualizar el PlatilloSeeder y RestaurantDetailSeeder.	Saúl M.	El seeder asigna platillos a los restaurantes de prueba correctamente.
HU_2.4: (Code) Corregir y actualizar el FaqSeeder y DatabaseSeeder principal.	Diego J.	El comando php artisan db:seed se ejecuta sin errores y puebla toda la BD.
HU_2.5: (Integración) Integrar el nuevo AuthController de Socialite con el existente.	Andrés J.	Se asegura que ambos flujos (email y Google) funcionen sin conflictos.
HU_2.6: (Test) Probar el nuevo flujo de auth y la ejecución de los seeders corregidos.	Alberto L.	Se confirma que php artisan migrate:fresh --seed funciona y se puede loguear con Google.

- Gestión de PRs (Sprint 2):
 - Fecha y hora del PR: Martes, 18 de Nov, 18:00h.
 - Responsable de hacer el PR: Cada integrante (a develop).
 - Responsable de aprobar el PR: Alberto L.
 - Rama destino: develop

Sprint 3: Sistema de reseñas (Restaurante y repartidor) (20–26 Nov)

Objetivo: Construir el módulo completo para que los clientes califiquen restaurantes y repartidores después de un pedido.

Historia de usuario (Tarea)	Asignación	Criterios de aprobación
HU_3.1: (Code) Crear migración, modelo y controlador para Reviews.	Saúl M.	El modelo Review y la migración create_reviews_table existen (con campos para pedido_id, rating_restaurante, rating_repartidor, etc.).
HU_3.2: (Code) Backend: Lógica para que un Cliente solo pueda calificar un pedido completado.	Saúl M.	ReviewController valida que el usuario sea el dueño del pedido y que el estado sea "Entregado".
HU_3.3: (Code) Frontend (Cliente): Mostrar botón "Calificar" en pedidos "Entregados".	Diego J.	En cliente/pedidos.blade.php, aparece un botón o link para calificar.
HU_3.4: (Code) Frontend (Cliente): Crear el formulario/modal para enviar las 2 calificaciones.	Diego J.	Un modal permite al cliente dar estrellas (1-5) y un comentario al restaurante y al repartidor.
HU_3.5: (Code) Frontend (Restaurante): Mostrar calificación promedio y lista de reseñas.	Eduardo L.	El dashboard del restaurante ahora muestra su rating promedio.
HU_3.6: (Code) Frontend (Repartidor): Mostrar calificación promedio y lista de reseñas.	Jorge A.	El dashboard del repartidor ahora muestra su rating promedio.
HU_3.7: (Test) Probar el flujo completo: Cliente pide > Pedido se entrega > Cliente califica > Ratings se actualizan.	Alberto L.	Se confirma que las calificaciones promedio cambian.

- Gestión de PRs (Sprint 3):
 - Fecha y hora del PR: martes, 25 de Nov, 18:00h.
 - Responsable de hacer el PR: Cada integrante (a develop).
 - Responsable de aprobar el PR: Eduardo L.
 - Rama destino: develop

Sprint 4: Implementación de PayPal y API REST (27 Nov–01 Dic)

Objetivo: Reemplazar el pago simulado por PayPal Sandbox y construir la API REST requerida por el profesor.

Historia de usuario (Tarea)	Asignación	Criterios de aprobación
HU_4.1: (Code) Implementar Backend de Pago con PayPal Sandbox.	Diego J.	Se crea un PaypalController que maneja la creación y captura de órdenes con la API de PayPal.
HU_4.2: (Code) Integrar Botón de Pago de PayPal en el frontend del carrito.	Saúl M.	El botón de PayPal reemplaza el formulario de pago simulado.
HU_4.3: (Integración) Actualizar PedidoController para que use la respuesta de PayPal (HU_4.1) antes de crear el pedido.	Andrés J.	El pedido solo se crea en la BD si el pago de PayPal es exitoso.
HU_4.4: (API) Implementar API de Autenticación (/api/login, /api/auth/google) con Sanctum.	Diego J.	Los endpoints funcionan y devuelven un token de API.
HU_4.5: (API) Implementar API de Pedidos (GET, POST) y Reseñas (GET) protegidos.	Saúl M.	Un usuario autenticado con token puede crear un pedido (con pago PayPal) y ver sus reseñas vía API.
HU_4.6: (API) Implementar API de Platillos (GET públicos).	Saúl M.	Cualquiera puede consultar la lista de restaurantes y platillos vía API.
HU_4.7: (Doc) Crear la Documentación de la API (Postman o README).	Alberto L.	Se entrega un documento que explica cómo usar todos los endpoints de la API.
HU_4.8: (Test) Prueba final E2E de todo el flujo (Google Login > Pedido > Pago PayPal > Calificación).	Eduardo G., Jorge A.	El equipo original valida la integración de todas las nuevas funcionalidades.

- Gestión de PRs (Sprint 4 - Final):
 - Fecha y hora del PR: Domingo, 30 de Nov, 20:00h.
 - Responsable de hacer el PR: Cada integrante (a develop).
 - Responsable de aprobar el PR: Jorge A.
 - Rama destino: main (Esta es la fusión final para la entrega).